实 验 报 告

姓名： 姚嫣如　　课程：离散数学Ⅱ
专业班级：23 网安　　实验时间：2025/12/15
上课老师：盛艳秀
学号：21080012078

（一） 实验题目 ：无人机巡检路径规划

（二） 实验目的

1、理解哈密顿图的定义
2、认识哈密顿图判定的困难性
3、实现哈密顿回路的查找近似算法
4、实现图和回路的可视化
5、了解 NP 完全问题的特性

（三） 实验要求

某大型太阳能发电站需要使用无人机定期巡检光伏板阵列。无人机需要访问每个检测点拍照检查，然后返回起降平台。
1、无人机可以在任意两个监测点之间飞行
2、必须访问每个检测点恰好一次
3、无人机电池续航有限，需要最短路径
4、典型的哈密顿回路问题（TSP 变体）

根据上述实验背景，需要完成以下实验需求：
1、以图模型描述问题：对无人机巡检光伏板阵列进行建模，得到一个带权完全图，每条边的权值是两检测点之间的距离。
2、任意选择一种近似算法，给出一个近似最优解，并可视化图示。
*非负权的带权图指图中的每条边都有一个非负数值。在一个具有非负权的带权完全图 G 中，一个回路的权值是该回路上所有边的权值之和。所有哈密顿回路中权值最小的哈密顿回路是最优回路

（四） 实验内容和实验步骤；

一．需求分析
1、本实验旨在解决无人机巡检光伏板阵列时的路径规划问题，该问题属于经

典的旅行商问题（TSP）变体。程序需在给定的带权完全图中寻找一条权值最小（或近似最小）的哈密顿回路。

（1）输入形式

① 节点数量 n：一个整数，代表监测点的个数。

② 距离矩阵 A：一个 n x n 的二维矩阵。矩阵元素 A[i][j]表示第 i 个监测点到第 j 个监测点之间的物理距离（权值）。由于是无向图，矩阵关于主对角线对称，且对角线元素为 0。

（2）输出的形式

- 图形化输出：

① 初始图：可视化显示带权完全图，包含所有节点和潜在的连接边。

② 结果图：在图中高亮显示计算出的哈密顿回路（巡检路径），用特定颜色连接各点形成闭环。

- 文本输出：

① 近似解：输出近似算法求得的节点访问顺序（路径）及其总权值。

② 精确解：对于小规模输入，输出暴力搜索求得的全局最优路径及总权值。

③ 结果对比：输出近似解相对于精确解的误差率分析。

（3）数据约束

① 节点数 n：对于精确解（特解）的验证，为了防止电脑运行时间过长，节点数规定限制在小规模（建议 n=6 或 n=7）；对于近似解，节点数 n 可以较大（如 n > 20）。

② 权值：非负实数，代表两点间的距离。

（4）程序所能实现的功能

① 建模与输入：能够读取距离矩阵，在内存中构建带权完全图模型。

② 可视化展示：利用绘图库将抽象的矩阵数据转换为直观的几何图形（自动计算节点坐标）。

③ 近似路径计算：针对任意规模数据，利用近似算法快速给出一条可行的哈密顿回路。

④ 精确路径计算：针对小规模数据（N 小于等于 6），利用暴力搜索算法找出理论上的最短路径。

二、概要设计

说明用到的数据结构定义、主程序的流程及各程序模块之间的调用关系。

1、数据结构定义

- 图模型存储：

  变量：A

  类型：numpy.ndarray

  定义：一个 N×N 的二维整数数组。A[i][j]存储节点 i 到节点 j 的距离权值。矩阵关于主对角线对称，且对角线元素为 0。

- 路径存储：

  变量名：path

  类型：List[int] (列表)

  定义：按顺序存储访问过的节点编号（例如 [0, 3, 1, 2, 0]）。

- 访问数组：

  变量名：visited

  类型：List[bool] (布尔列表)

  定义：长度为 N 的布尔数组，索引对应节点编号，True 表示已访问，False 表示未访问。

- 递归全局变量：

  min_cost (int)：记录当前发现的全局最小路径总权值。

  best_path (列表)：记录对应最小权值的路径序列。

- 可视化辅助：

  使用 networkx.Graph 对象存储图结构，使用字典 pos 存储节点在绘图区的 (x, y) 坐标（通过 circular_layout 生成）。

2. 主程序流程

(1) 输入阶段：程序启动，提示用户输入监测点数量 N。进行输入合法性校验（必须为整数且 N >= 2）。

(2) 生成带权完全图：调用 random_A(n) 生成满足无向图性质（对称）的随机邻接矩阵，并打印。

(3) 初始可视化：调用 visualize_initial，绘制带有所有边权值的初始完全图。

(4) 近似解计算：

调用 approx_solution 执行最近邻算法。

输出近似路径及总距离。

调用 visualize_solution 绘制近似路径图。

(5) 精确解计算（条件执行）：

判断 N <= 14（防止计算超时）。

若满足，调用 exact_solution 执行 DFS 暴力搜索。
输出精确路径及总距离，并计算近似解的误差率。
调用 visualize_solution 绘制精确路径图。
若不满足，打印提示跳过此步骤。

3. 模块调用关系

main (主控模块)：负责全局调度。
→ 调用 random_A：生成数据。
→ 调用 visualize_initial：绘图。
→ 调用 approx_solution：求解近似解。
→ 调用 exact_solution：求解精确解。
→ exact_solution 内部嵌套定义并调用 dfs：执行递归搜索。
→ 调用 visualize_solution：绘制结果路径。

三、详细设计

```
import numpy as np    # 处理矩阵数组
import matplotlib
matplotlib.use('TkAgg') # 或者 'Qt5Agg'，强制指定后端
import matplotlib.pyplot as plt    # 画图展示
import networkx as nx    # 辅助画图



# 根据顶点数，随机生成邻接矩阵。
def random_A(n):
    A = np.zeros((n, n), dtype=int)    #初始化一个 nxn 的全 0 矩阵，值为 int 类型

    # 遍历矩阵，填充矩阵的右上角（无向图，左下角和右上角是对称的）。
    for i in range(n):
        # 对角线(i==j)为 0，j 从 i+1 开始
        for j in range(i + 1, n):
            # 随机生成一个 5 到 50 之间的整数作为距离
            weight = np.random.randint(5, 50)

            # A→B = B→A
```

```
                A[i][j] = weight
                A[j][i] = weight

        return A



# 计算近似解算法 (最近邻算法)
# 在没去过的邻居中，每次都选最近的一个走。
def approx_solution(n, A):
        visited = [False] * n      # 访问数组
        path = []      # 记录走过的路径
        cost = 0          # 记录总权重

        # 从 0 号节点出发
        current = 0
        visited[0] = True
        path.append(0)

        # 循环找下一个点
        for _ in range(n - 1):
                # 初始化
                minn = float('inf')    # 初始化无穷大
                next_node = -1    # 还没找到下一个点

                # 遍历邻居
                for j in range(n):
                        # 更新找到最小距离邻居
                        if not visited[j] and A[current][j] < minn:
                                minn = A[current][j]    # 更新最小距离
                                next_node = j    # 锁定这个节点

                # 加入路径
                if next_node != -1:
                        visited[next_node] = True
```

```
                path.append(next_node)
                cost += minn
                current = next_node

        # 从终点回到起点 0，形成闭环
        cost += A[current][0]
        path.append(0)

        return path, cost



# 精确解算法 (暴力搜索 DFS )

# 定义全局变量
best_path = []
min_cost = float('inf')


# 暴力穷举：递归函数，尝试走完所有的路，走到头了就算算总距离，如果比记录的最好成绩还好，就更新记录。
def exact_solution(n, A):
    global min_cost, best_path

    # 重置变量
    min_cost = float('inf')
    best_path = []

    # 访问数组
    visited = [False] * n
    visited[0] = True

    # 内部递归函数
    # u: 当前顶点, count: 总访问过顶点数, cost: 当前总权重, path: 当前路径
    def dfs(u, count, current_cost, current_path):
```

```
    global min_cost, best_path

    # 如果当前权重已经大于之前的最优解，没有必要再递归
    if current_cost >= min_cost:
        return

    # 所有点都去过了
    if count == n:
        # 终点 → 顶点
        total = current_cost + A[u][0]
        # 更新最优解
        if total < min_cost:
            min_cost = total
            best_path = current_path[:] + [0]   # 记下来
        return

    # 循环遍历所有没去过的邻居
    for v in range(n):
        if not visited[v]:
            visited[v] = True
            current_path.append(v)

            # 下一层
            dfs(v, count + 1, current_cost + A[u][v], current_path)

            # 回溯，清空当前临时路径
            current_path.pop()
            visited[v] = False

# 启动递归
dfs(0, 1, 0, [0])

return best_path, min_cost
```

```
# 可视化模块

# 因为没有坐标(x,y)，只有距离,为了画图,把点均匀地排成圆形布局）。
def get_layout(n):
    """
    因为我们没有坐标(x,y)，只有距离。
    为了画图好看，我们把点均匀地排成一个圆圈（圆形布局）。
    """
    G_temp = nx.complete_graph(n)
    return nx.circular_layout(G_temp)


# 可视化初始邻接矩阵，显示所有的边和权值。
def visualize_initial(n, A):
    G = nx.Graph()
    G.add_nodes_from(range(n))

    # 把矩阵里的数据变成图里的边
    for i in range(n):
        for j in range(i + 1, n):
            # 将边的权重加进去
            G.add_edge(i, j, weight=A[i][j])

    pos = get_layout(n)

    plt.figure(figsize=(6, 6))  # 创建一个正方形的画板

    # 画点
    nx.draw_networkx_nodes(G, pos, node_color='lightgray', node_size=600)
    nx.draw_networkx_labels(G, pos, font_weight='bold')

    # 画边（虚线表示潜在的路径）
    nx.draw_networkx_edges(G, pos, edge_color='gray', style='dashed',
alpha=0.5)
```

```
    # 画边上的数字（权值）
    # 如果点太多，数字会挤在一起，所以只有点少的时候才画数字
    #if n <= 8:
    edge_labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=8)

    plt.title(f"Initial Random Graph (N={n})")
    plt.axis('off')  # 关掉坐标轴刻度
    plt.show()       # 弹窗显示


# 可视化计算结果，用红实线标出路径。
def visualize_solution(n, A, path, title):
    G = nx.Graph()
    G.add_nodes_from(range(n))
    pos = get_layout(n)  # 使用和初始时一样的布局，保证点的位置不动

    plt.figure(figsize=(6, 6))

    # 画背景点
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=600)
    nx.draw_networkx_labels(G, pos)

    # 根据结果 path 列表生成边
    path_edges = []
    path_weights = {}
    for i in range(len(path) - 1):
        u = path[i]
        v = path[i + 1]
        path_edges.append((u, v))
        path_weights[(u, v)] = A[u][v]

    # 画红线
```

```
    nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='red', width=2)

    # 在红线上标出距离
    nx.draw_networkx_edge_labels(G, pos, edge_labels=path_weights, font_color='red')

    plt.title(title)
    plt.axis('off')
    plt.show()


# 主函数
def main():
    print("========= 无人机巡检路径规划 =========")

    # 获取顶点数
    try:
        nn = input("请输入监测点数量 N : ")
        n = int(nn)
        if n < 2:
            print("至少需要 2 个点，请重新输入!! ")
            exit()
    except ValueError:
        print("输入错误，请输入整数!")
        exit()

    # 随机生成距离矩阵
    A = random_A(n)

    print(f"\n========== 随机生成 {n} 个顶点的距离矩阵... ==========")
    print("生成的邻接矩阵如下:")
    print(A)

    # 可视化初始距离矩阵
```

```
    visualize_initial(n, A)

    # 近似算法
    print(f"\n========== 计算 近似解 (最近邻) ==========")
    path1, cost1 = approx_solution(n, A)
    print(f"近似路径: {path1}")
    print(f"近似总距离: {cost1}")

    visualize_solution(n, A, path1, f"Approximate Result (Cost: {cost1})")

    # 精确算法 (仅当 N <=  时)
    if n <= 14:
        print(f"\n========== 计算 精确解  (暴力搜索) ==========")
        path2, cost2 = exact_solution(n, A)
        print(f"精确路径: {path2}")
        print(f"精确总距离: {cost2}")

        # 计算误差率
        error_rate = (cost1 - cost2) / cost2 * 100
        print(f"近似算法误差率: {error_rate:.2f}%")

        visualize_solution(n, A, path2, f"Exact Result (Cost: {cost2})")
    else:
        print("\n========== 当前顶点数大于 6，将跳过精确解计算（防止电脑卡死）==========")

    print("\n========== 实验结束 ==========")

if __name__ == "__main__":
    main()
```

四、调试分析

1、调试过程中所遇到的问题及解决方法；

（1）问题：图形化界面无法显示

- 现象：程序运行结束后，命令行提示完成，但没有弹出图形窗口。
- 解决方法：这是由于 Matplotlib 后端配置问题。在代码头部添加了 matplotlib.use('TkAgg') 强制指定后端，并确保在绘图代码末尾调用 plt.show()，成功解决了弹窗问题。

（2）问题：精确解递归中无法更新最优结果
- 现象：在 dfs 函数内部更新了 min_cost，但递归结束后外部打印的 min_cost 仍为初始无穷大。
- 解决方法：在 dfs 函数内部首行添加 "global min_cost, best_path" 声明，明确告知解释器需要修改外部定义的全局变量，而非创建局部变量。

2、算法的时空分析；

（1） 近似算法 (最近邻法)：
- 时间复杂度：O(N^2)。我们需要访问 N 个节点，对于每个节点，都需要遍历查找 N 个邻居中的最小值。计算速度极快，即使 N=1000 也能瞬间完成。
- 空间复杂度：O(N^2)。主要用于存储邻接矩阵。

（2） 精确算法 (DFS 暴力搜索)：
- 时间复杂度：O(N!)。TSP 问题的解空间是所有顶点的全排列。随着 N 的增加，计算量呈阶乘级爆炸式增长。
- N=10 时约为 360 万次计算，尚可接受。
- N=14 时，计算量达到千亿级别，运行时间会显著增加。
- 空间复杂度：O(N^2)用于存储矩阵，O(N)用于递归栈的深度。

（五） 实验结果

列出对于给定的输入所产生的输出结果。若可能，测试随输入规模的增长所用算法的实际运行时间的变化。

1、顶点数 N = 5

```
========== 无人机巡检路径规划 ==========
请输入监测点数量 N : 5

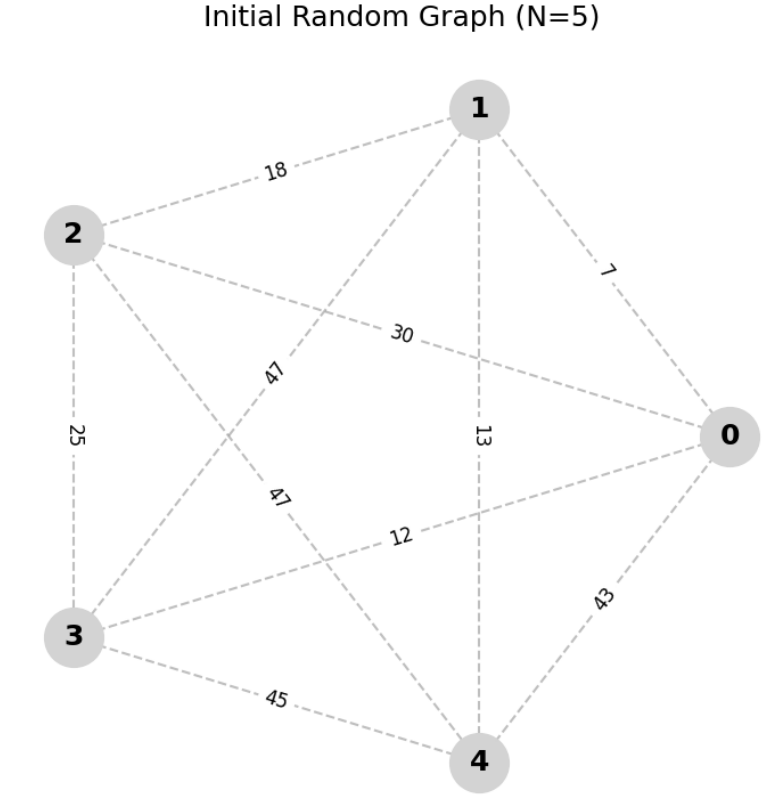
========== 随机生成 5 个顶点的距离矩阵... ==========
生成的邻接矩阵如下:
[[ 0  7 30 12 43]
 [ 7  0 18 47 13]
 [30 18  0 25 47]
 [12 47 25  0 45]
 [43 13 47 45  0]]

========== 计算 近似解 (最近邻) ==========
近似路径: [0, 1, 4, 3, 2, 0]
近似总距离: 120

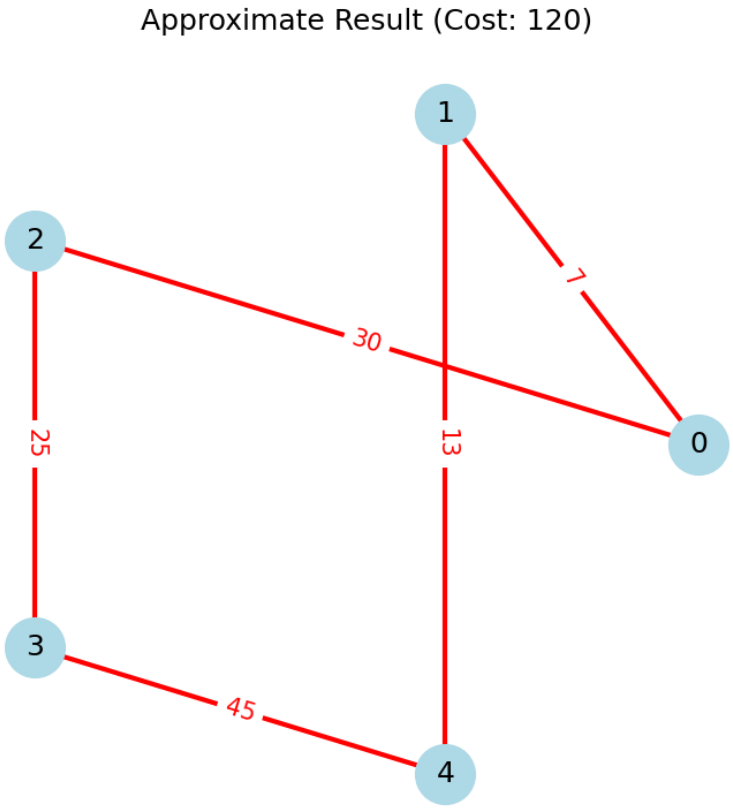
========== 计算 精确解 (暴力搜索) ==========
精确路径: [0, 1, 4, 2, 3, 0]
精确总距离: 104
近似算法误差率: 15.38%

========== 实验结束 ==========
```

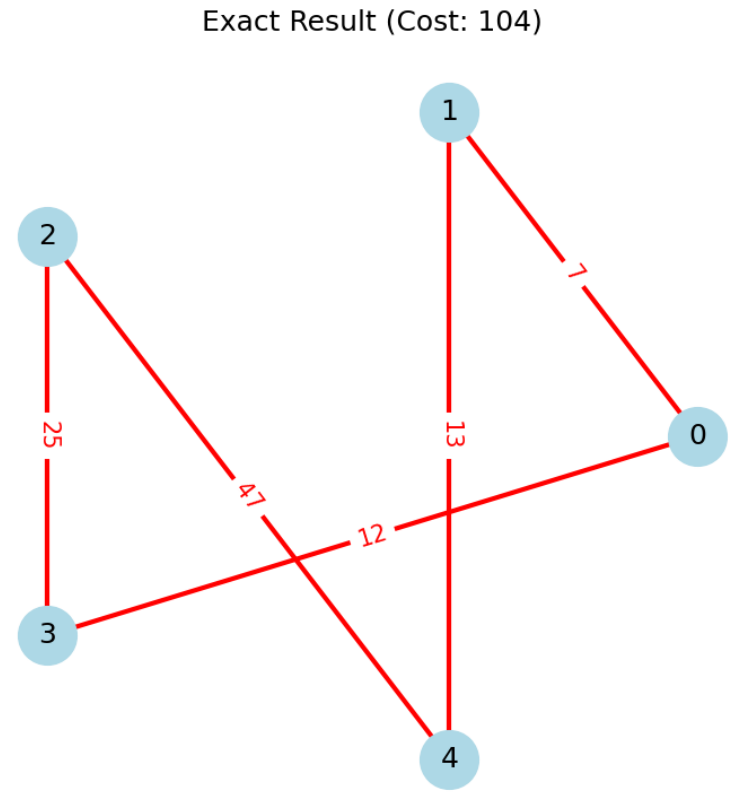
初始带权完全图：



近似解路径图：



精确解路径图：



2、顶点数 N = 13

```
========== 无人机巡检路径规划 ==========
请输入监测点数量 N : 13

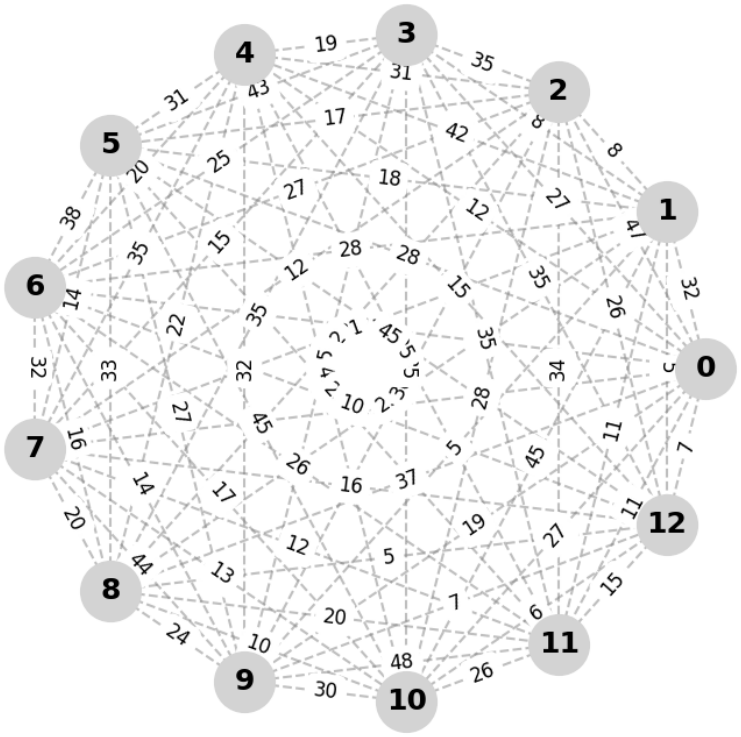
========== 随机生成 13 个顶点的距离矩阵... ==========
生成的邻接矩阵如下:
[[ 0 32 47 27 12 28 13 47 37 19 27 11  7]
 [32  0  8  8 42 18 28 31 29  5 45 11  5]
 [47  8  0 35 31 17 27 12 12 30 28 34 26]
 [27  8 35  0 19 43 25 15 35 35 35 35 35]
 [12 42 31 19  0 31 20 35 22 32 49 25 15]
 [28 18 17 43 31  0 38 14 33 27 45 29 45]
 [13 28 27 25 20 38  0 32 16 14 17 26 10]
 [47 31 12 15 35 14 32  0 20 44 13 12 16]
 [37 29 12 35 22 33 16 20  0 24 10 20  5]
 [19  5 30 35 32 27 14 44 24  0 30 48  7]
 [27 45 28 35 49 45 17 13 10 30  0 26  6]
 [11 11 34 35 25 29 26 12 20 48 26  0 15]
 [ 7  5 26 35 15 45 10 16  5  7  6 15  0]]

========== 计算 近似解 (最近邻) ==========
近似路径: [0, 12, 1, 9, 6, 8, 10, 7, 2, 5, 11, 4, 3, 0]
近似总距离: 199

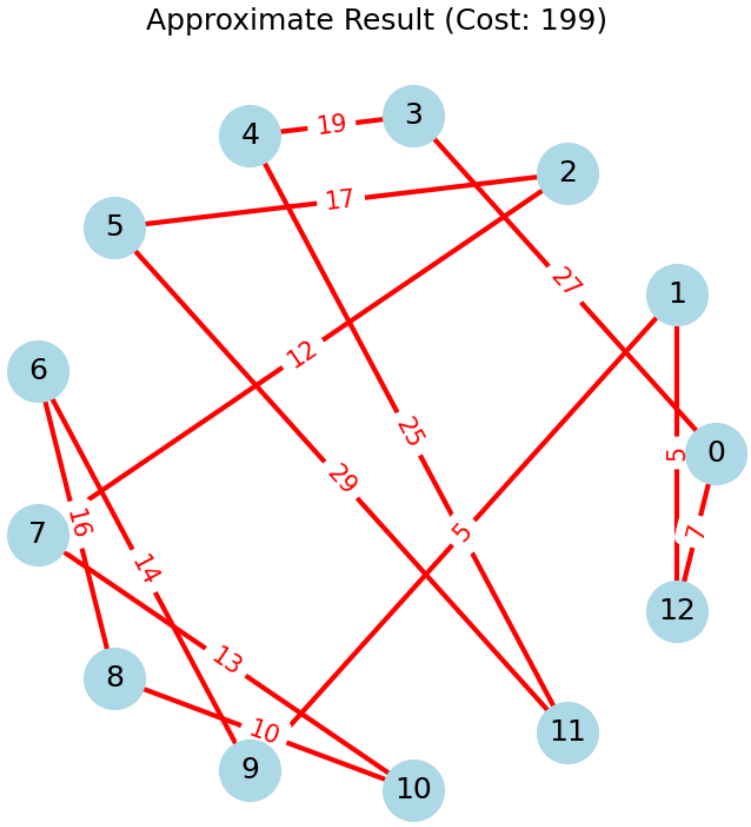
========== 计算 精确解 (暴力搜索) ==========
精确路径: [0, 4, 3, 1, 9, 6, 12, 10, 8, 2, 5, 7, 11, 0]
精确总距离: 150
近似算法误差率: 32.67%

========== 实验结束 ==========
```

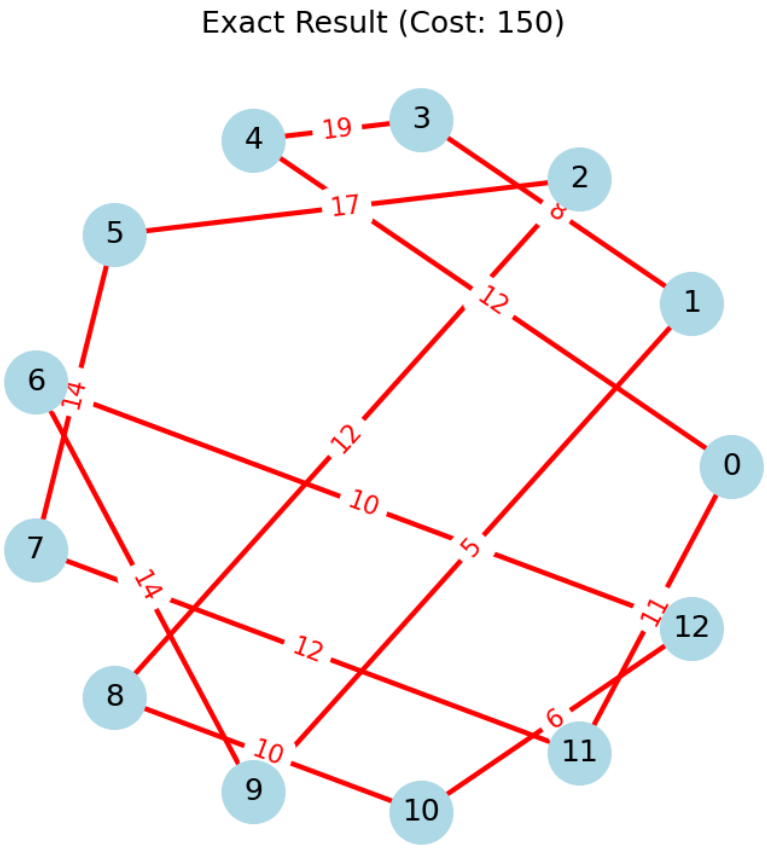
初始带权完全图：



近似解路径图：



精确解路径图：



（六） 实验总结

通过本次实验，我深刻理解了图论在计算机科学中的实际应用，了解了旅行商问题（TSP）作为 NP-Hard 问题的计算难度。通过编写代码，直观地看到了 O(N^2) 的贪心算法与 O(N!) 的暴力搜索算法在运行效率上的巨大鸿沟。虽然近似算法（最近邻）不能保证得到全局最优解，但其极高的效率使其在工程实践中处理大规模数据时具有重要价值。

在实现 DFS 回溯算法时，深入理解了“递归”与“回溯”的机制，特别是如何在递归返回时“恢复现场”（即 visited[v] = False 和 path.pop()）。同时，掌握了 Python 中 global 关键字在嵌套函数中的作用，避免了变量作用域导致的逻辑错误。

实验最后的可视化环节，利用 Matplotlib 和 NetworkX 将枯燥的邻接矩阵转化为直观的图结构，不仅验证了数据的正确性（如对称性），还能清晰地展示出路径的走向，帮助快速发现算法逻辑中的潜在错误（如路径是否闭环、

是否存在交叉等）。